Kasi Yedumati · Full Stack Training

# Querying with SQL

**Module Outcome:**

- Filter Swiggy products with WHERE, comparison, LIKE, and IN
- Sort and limit so a catalogue page is not 10,000 rows
- Write a SELECT you could paste into Workbench and trust

**Note:** This chapter uses **MySQL** syntax. String literals use single quotes (`'value'`). For homework rows, load `labs/training_db_seed.sql` first.

## Comparison Operators

In a typical e-commerce scenario, users would generally filter the products with good ratings, or want to purchase the products of a certain brand or of a certain price.

The database contains a **product** table that stores the data of products like name, category, price, brand and rating.

| Operator | Description |
|---|---|
| `=` | Equals to |
| `<>` | Not equals to |
| `<` | Less than |
| `<=` | Less than or equal to |
| `>` | Greater than |
| `>=` | Greater than or equal to |

**Examples:**

1. Get all the details of the products whose category is 'food' from the product table.

```
SELECT
    *
FROM
    product
WHERE
    category = 'Food';
```

**Output:**

| name | category | price | brand | rating |
|---|---|---|---|---|
| Chocolate cake | Food | 25 | Britannia | 3.7 |
| Strawberry cake | Food | 60 | Cadbury | 4.1 |
| Chocolate cake | Food | 60 | Cadbury | 2.5 |

| name | category | price | brand | rating |
|---|---|---|---|---|
| ... | ... | ... | ... | ... |

2. Get all the details of the products that does not belong to Food category from the product table.

```
SELECT
    *
FROM
    product
WHERE
    category <> 'Food';
```

**Output:**

| name | category | price | brand | rating |
|---|---|---|---|---|
| Blue shirt | clothing | 750 | Denim | 3.8 |
| Blue Jeans | clothing | 200 | puma | 3.6 |
| Black jeans | clothing | 750 | Denim | 4.5 |
| ... | ... | ... | ... | ... |

Similarly, we can use other comparison operators like greater than ( `>` ), greater than or equal to ( `>=` ), less than ( `<` ), less than or equal to ( `<=` ) to filter the data as per the requirement.

## IS NULL and IS NOT NULL

`NULL` represents missing or unknown data. We cannot use `= NULL` to check for NULL values — we must use `IS NULL` or `IS NOT NULL`.

| Operator | Description |
|---|---|
| `IS NULL` | True when the value is NULL |
| `IS NOT NULL` | True when the value is not NULL |

**Syntax:**

```
SELECT
    *
FROM
    table_name
WHERE
    column_name IS NULL;
```

**Example 1:** Get all products that have no brand specified.

```
SELECT
    name,
    category,
    price,
```

```
    brand
FROM
    product
WHERE
    brand IS NULL;
```

**Output:**

| name | category | price | brand |
|---|---|---|---|
| Generic Notebook | stationery | 50 | NULL |
| Plain T-shirt | clothing | 299 | NULL |

**Example 2:** Get all products that have a brand specified.

```
SELECT
    name,
    brand,
    rating
FROM
    product
WHERE
    brand IS NOT NULL
ORDER BY
    rating DESC;
```

**Note:** `IS NULL` is commonly used with `LEFT JOIN` to find rows with no match in the joined table (see Joins).

## String Operators

## LIKE Operator

`LIKE` operator is used to perform queries on strings. This operator is especially used in `WHERE` clause to retrieve all the rows that match the given pattern.

We write **patterns** using the following **wildcard characters**:

| Symbol | Description | Example |
|---|---|---|
| `%` | Represents zero or more characters | `ch%` finds ch, chips, chocos... |
| `_` | Represents a single character | `_at` finds mat, hat, bat... |

**Common Patterns:**

| Pattern | Example | Description |
|---|---|---|
| Exact Match | `WHERE name LIKE 'Mobiles'` | Retrieves products whose name is exactly equals to 'mobiles' |

| Pattern | Example | Description |
| --- | --- | --- |
| Starts with | `WHERE name LIKE 'mobiles%'` | Retrieves products whose name starts with 'mobiles' |
| Ends with | `WHERE name LIKE '%mobiles'` | Retrieves products whose name ends with 'mobiles' |
| Contains | `WHERE name LIKE '%mobiles%'` | Retrieves products whose name contains 'mobiles' |
| Pattern Matching | `WHERE name LIKE 'a_%'` | Retrieves products whose name starts with 'a' and have at least 2 characters in length |

**Syntax:**

```
SELECT
    *
FROM
    table_name
WHERE
    C1 LIKE matching_pattern;
```

**Examples:**

1. Get all the products in the 'Gadgets' category from the product table.

```
SELECT
    *
FROM
    product
WHERE
    category LIKE 'Gadgets';
```

**Output:**

| name | Category | price | brand | rating |
| --- | --- | --- | --- | --- |
| Smart Watch | Gadgets | 17000 | Apple | 4.9 |
| Smart Cam | Gadgets | 2600 | Realme | 4.7 |
| Smart TV | Gadgets | 40000 | Sony | 4.0 |
| Realme Smart Band | Gadgets | 3000 | Realme | 4.6 |

2. Get all the products whose name starts with 'Bourbon' from the product table.

```
SELECT
    *
FROM
    product
WHERE
    name LIKE 'Bourbon%';
```

Here `%` represents that, following the string 'Bourbon', there can be 0 or more characters.

**Output:**

| name | Category | price | brand | rating |
|---|---|---|---|---|
| Bourbon Small | food | 10 | Britannia | 3.9 |
| Bourbon Special | food | 15 | Britannia | 4.6 |
| Bourbon with extra cookies | food | 30 | Britannia | 4.4 |

3. Get all smart electronic products i.e., name contains 'smart' from the product table.

```
SELECT
    *
FROM
    product
WHERE
    name LIKE '%smart%';
```

Here `%`, before and after the string represents that there can be 0 or more characters succeeding or preceding the string.

**Output:**

| name | Category | price | brand | rating |
|---|---|---|---|---|
| Smart Watch | Gadgets | 17000 | Apple | 4.9 |
| Smart Cam | Gadgets | 2600 | Realme | 4.7 |
| Smart TV | Gadgets | 40000 | Sony | 4.0 |
| Realme Smart Band | Gadgets | 3000 | Realme | 4.6 |

4. Get all the products which have exactly 5 characters in brand from the product table.

```
SELECT
    *
FROM
    product
WHERE
    brand LIKE '_____';
```

**Output:**

| name | category | price | brand | rating |
|---|---|---|---|---|
| Blue Shirt | clothing | 750 | Denim | 3.8 |
| Black Jeans | clothing | 750 | Denim | 4.5 |
| Smart Watch | Gadgets | 17000 | Apple | 4.9 |

| name | category | price | brand | rating |
|---|---|---|---|---|
| ... | ... | ... | ... | ... |

**Note:**

- The percent sign (`%`) is used when we are not sure of the number of characters present in the string.
- If we know the exact length of the string, then the wildcard character underscore (`_`) comes in handy.

## Logical Operators

Based on logical operators, we can perform queries based on multiple conditions: **AND**, **OR**, **NOT**.

| Operator | Description |
|---|---|
| AND | Used to fetch rows that satisfy two or more conditions |
| OR | Used to fetch rows that satisfy at least one of the given conditions |
| NOT | Used to negate a condition in the WHERE clause |

**Syntax:**

```
SELECT
    *
FROM
    table_name
WHERE
    Condition1
    operator condition2
    operator condition3
    .....;
```

**Example 1:** Get all the details of the products whose category is 'clothing' and price less than equal to 1000 from the product table.

```
SELECT
    *
FROM
    product
WHERE
    category = 'clothing'
    AND price <= 1000;
```

**Output:**

| name | category | price | brand | rating |
|---|---|---|---|---|
| Blue Shirt | clothing | 750 | Denim | 3.8 |
| Blue Jeans | clothing | 800 | puma | 3.6 |
| Black Jeans | clothing | 750 | Denim | 4.5 |

| name | category | price | brand | rating |
|---|---|---|---|---|
| ... | ... | ... | ... | ... |

**Example 2:** Ignore all the products with name containing 'cake' from the list of products.

```
SELECT
    *
FROM
    product
WHERE
    NOT name LIKE '%cake%';
```

## Multiple Logical Operators

We can also use the combinations of logical operators to combine two or more conditions. These compound conditions enable us to fine-tune the data retrieval requirements.

**Precedence:** When a query has multiple operators, operator precedence determines the sequence of operations.

**Order of precedence (high to low):**

1. NOT
2. AND
3. OR

**Example:** Fetch the products that belongs to Redmi brand and rating greater than 4 or the products from Oneplus brand.

```
SELECT
    *
FROM
    product
WHERE
    brand = 'Redmi'
    AND rating > 4
    OR brand = 'Oneplus';
```

In the above query, AND has the precedence over OR. So, the above query is equivalent to:

```
SELECT
    *
FROM
    product
WHERE
    (brand = 'Redmi'
    AND rating > 4)
    OR brand = 'Oneplus';
```

## IN and BETWEEN Operators

**Quick Tip:** It is suggested to always use parenthesis to ensure correctness while grouping the conditions.

## IN Operator

Retrieves the corresponding rows from the table if the value of column (C1) is present in the given values (v1, v2, ...).

**Syntax:**

```
SELECT
    *
FROM
    table_name
WHERE
    C1 IN (v1, v2, ...);
```

**Example:** Get the details of all the products from product table, where the brand is either 'puma', 'Mufti', 'Levi's', 'Lee' or 'Denim'.

```
SELECT
    *
FROM
    product
WHERE
    brand IN ('puma', 'Levi''s', 'Mufti', 'Lee', 'Denim');
```

## BETWEEN Operator

Retrieves all the rows from table that have column (c1) value present between the given range (v1 and v2).

**Syntax:**

```
SELECT
    *
FROM
    table_name
WHERE
    c1 BETWEEN v1
    AND v2;
```

**Note:** BETWEEN operator is inclusive, i.e., both the lower and upper limit values of the range are included.

**Example:** Find the products with price ranging from 1000 to 5000.

```
SELECT
    name,
    price,
    brand
FROM
    product
WHERE
    price BETWEEN 1000
    AND 5000;
```

**Output:**

| name | price | brand |
|---|---|---|
| Blue shirt | 1000 | puma |
| Smart cam | 2600 | Realme |
| Realme Smart Band | 3000 | Realme |

## Possible Mistakes with BETWEEN

1. When using the BETWEEN operator, the first value should be less than second value. If not, we'll get an incorrect result depending on the DBMS.

```
SELECT name, price, brand
FROM product
WHERE price BETWEEN 500 AND 300;
```

**Output:** (empty — no results)

2. We have to give both lower limit and upper limit while specifying range.

```
SELECT name, price, brand
FROM product
WHERE price BETWEEN AND 300;
```

**Output:** Error: You have an error in your SQL syntax (near `AND` at line 1)

3. The data type of the column for which we're using the BETWEEN operator must match with the data types of the lower and upper limits.

```
SELECT name, price, brand
FROM product
WHERE name BETWEEN 300 AND 500;
```

**Output:** (empty)

## ORDER BY and DISTINCT

## ORDER BY

We use ORDER BY clause to order rows. By default, ORDER BY sorts the data in the ascending order.

**Syntax:**

```
SELECT
    column1,
    column2,
```

```
    ... columnN,
FROM
    table_name [WHERE condition]
ORDER BY
    column1 ASC/DESC,
    column2 ASC/DESC;
```

**Example:** Get all products in order of lowest price and highest rating in 'puma' brand.

```
SELECT
    name,
    price,
    rating
FROM
    product
WHERE
    brand = 'puma'
ORDER BY
    price ASC,
    rating DESC;
```

**Output:**

| name | price | rating |
|---|---|---|
| Black shirt | 600 | 4.8 |
| Blue jeans | 800 | 3.6 |
| Blue shirt | 1000 | 4.3 |

## DISTINCT

DISTINCT clause is used to return the distinct i.e., unique values.

**Syntax:**

```
SELECT
    DISTINCT column1,
    column2, ...
    columnN
FROM
    table_name
WHERE
    [condition];
```

**Example:** Get all the brands present in the product table.

```
SELECT
    DISTINCT brand
FROM
    product
ORDER BY
    brand;
```

**Output:**

| Brand |
| --- |
| Absa |
| Apple |
| ... |

## Pagination

Using pagination, only a chunk of the data can be sent to the user based on their request. And, the next chunk of data can be fetched only when the user asks for it.

We use **LIMIT** and **OFFSET** clauses to select a chunk of the results.

## LIMIT

LIMIT clause is used to specify the number of rows (n) we would like to have in result.

**Syntax:**

```
SELECT
    column1,
    column2,
    ... columnN
FROM
    table_name
LIMIT n;
```

**Example:**

```
SELECT
    name,
    price,
    rating
FROM
    product
WHERE
    brand = 'puma'
ORDER BY
    rating DESC
LIMIT 2;
```

**Output:**

| name | price | rating |
| --- | --- | --- |
| Black shirt | 600 | 4.8 |
| Blue shirt | 1000 | 4.3 |

**Note:** If the limit value is greater than the total number of rows, then all rows will be retrieved.

## OFFSET

OFFSET clause is used to specify the position (from (n+1)th row) from where the chunk of the results are to be selected.

**Syntax:**

```
SELECT
    column1,
    column2,
    ... columnN
FROM
    table_name
LIMIT m
OFFSET n;
```

**Example:** Get all the details of 5 top-rated products, starting from 7th row.

```
SELECT
    name,
    price,
    rating
FROM
    product
ORDER BY
    rating DESC
LIMIT 5
OFFSET 6;
```

**Output:**

| name | price | rating |
|---|---|---|
| Burbon Special | 15 | 4.6 |
| Realme Smart Band | 3000 | 4.6 |
| Harry potter and the Goblet of fire | 431 | 4.6 |
| Black Jeans | 750 | 4.5 |
| potato chips cream & onion | 63 | 4.5 |

## Possible Mistakes with OFFSET

1. Using OFFSET before the LIMIT clause.

```
SELECT * FROM product OFFSET 2 LIMIT 4;
```

**Output:** Error: You have an error in your SQL syntax (near `2` at line 1)

2. Using only OFFSET clause.

```
SELECT * FROM product OFFSET 2;
```

**Output:** Error: You have an error in your SQL syntax (near `2` at line 1)

**Note:** OFFSET clause should be placed after the LIMIT clause. Default OFFSET value is 0.